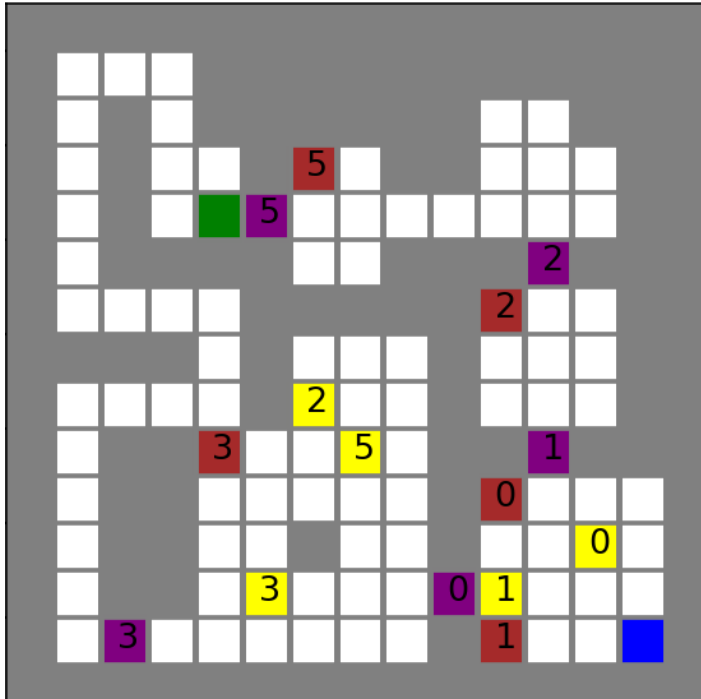


Home assignment 2: MDP

Introduction

The point of the exercise is to experiment with solving a problem using MDP principles that were taught in class.



The problem is similar to the problem in the previous assignment; the only change is that now we're in a stochastic environment. This assignment will ask you to create a policy using MDP principles since the environment/game will run itself only needing for every step the next action to be taken. This document will address the changes/additions that are relevant to this assignment.

Problem:

Input: in addition to the initial setup of the game, there is a dictionary of values that depict the environment behavior, and a maximum amount of steps. This will be expanded on later.

Possible actions: The policy you will write will output one of the four actions, however the game will not always succeed performing your chosen action (think of the ground being too slippery). The previous constraints still apply to the action **performed**.

Success: The agent finds a path to the goal.

Failure: No possible path to the goal can be found in the limited number of steps or dead-end.

Assignment:

For this assignment we've attached code for you to use. Most of what's needed is implemented. What you will have to do is implement the Controller class functions so the checker code can be utilized to solve the problem.

ex2.py file has the following functions for you to implement (you could also have helper functions as you see fit):

1. Choose_next_action function – receives a state of the game (which is a tuple of the game map, tuple of the agent position, the amount of steps taken, if the game is finished, if the game was successful) and returns a single action (e.g. “U”)
2. __init__ function – receives a pressure plate game and initializes the Controller class. This function does not need to be rewritten but you could create the policy already in the initialization.

DO NOT CHANGE THE FUNCTIONS NAMES OR INPUT!

Input:

In the check.py there will be the solve function that will receive the pressure plate problem and will create the policy controller according to ex2.py.

The pressure plate problem class is created by the function create_pressure_plate_game which receives a tuple of 4 values.

The values represent:

1. Max steps – the maximum amount of steps the policy will be run.
2. Map – the initial map of the problem.
3. Model – the environment model representing the probabilities and the rewards of the environment.
4. Debug mode – Boolean value to output the history of actions throughout a solve.

Model:

The environment model is represented by a dictionary.

Dictionary keys and what they hold:

‘chosen_action_prob’ – holds a dictionary of the possible actions and the probabilities of succeeding in the action or “slipping” to another action, which is then represented by a list of 4 values. The probabilities for each list are in this order “U”, “L”, “R”, “D”.

‘finished_reward’ – the reward to add if succeeded in reaching a goal.

‘opening_door_reward’ – holds a dictionary that for each key holds the reward to add if able to open doors of that key type.

‘step_punishment’ – the punishment to reduce from the final reward tally when finishing the game for each step.

‘seed’ – the seed value to use for randomizing values.

```
# ["U", "L", "R", "D"]
example = {'chosen_action_prob': {'U': [0.9, 0.05, 0.05, 0], 'L': [0.1, 0.8, 0.075, 0.025],
                                'R': [0.05, 0.05, 0.85, 0.05], 'D': [0.05, 0.1, 0.15, 0.7]},
          'finished_reward': 150,
          'opening_door_reward': {10: 5, 11: 7, 12: 9, 13: 11, 14: 13, 15: 15, 16: 17, 17: 19, 18: 21, 19: 23},
          'step_punishment': -1,
          'seed': 42}
```

Notes:

- You may change the code in `pressure_plate.py` but make sure to not rely on it to create your policy, since it will not be submitted.
- You can use any of the functions of the `pressure_plate` class on the “original_game” **except** the `submit_next_action` and `_open_doors` functions since it will distort the performance.
- If you wish to use `submit_next_action` and `_open_doors` please create a deep copy of the “original_game”.
- It is advised to create multiple different environment models for the problems you test. The `finished_reward` can be assumed to be greater than `max_steps`, `opening_door_reward` can be assumed to be lower than `finished_reward` (and can be negative), and step punishment can be assumed to be lower than `finished_reward/max_steps` but always negative.
- Please be consistent with your representations (actions and states, e.g. capitalized letters, underscores, etc.). The testing is automatic and even if the code is compiled correctly you will lose points for inconsistencies.
- Please make sure that your code works on both Linux and Windows, and please ask permission in the assignment forum to use any non-built-in libraries/packages (any library/package that you have to perform pip install to use it).

Grading:

Your policy does not have to be optimal if computationally impossible, but it must be on average better than the random policy, sub-par policy (which is unknown to you) and on-par policy (which is unknown to you).

The assignment will be checked automatically, and it is important that the correct names of the files, classes and functions are kept.

The grading program will consist of 20 different “PressurePlate” problems of increasing difficulty, each with a different environment model. Your policy must pass each problem under 5 minutes. If you cannot finish the problems in under 5 minutes the best problems will be taken, and the rest will be considered as failures (standing in place – reward = `max_steps * step_punishment`) and their will be added to your average.

Your code will need to perform better than the on-par policy on **average** to get 100. Meaning that the average reward of your policy on the 20 different problems with different environment models, will need to be higher than the on-par policy to get 100.

Policies that do not pass the on-par policy will get a lower grade based on the distance to the on-par average. Policies that do not pass the sub-par policy but pass the random policy will get a grade between 60 – 80. Policies that do not pass the random policy will receive 0.

You can assume that maximal matrix size is 20x20 and that at most there will be 10 key blocks.

NOTE: If anyone finds any bugs (if any) in the `pressure_plate` code in terms of incorrect performance of the game procedure, the first to upload to the assignment forum and suggest a correct fix will receive 5 bonus points.

Self-check: The attached file check.py has two example problems. It will not check the correctness of code or is enough for sufficient testing. It is encouraged to create more example problems to check the robustness of your code.

Attached files:

The files include:

1. Ex2.py – The only file to be submitted and the only one relevant for grading.
2. check.py – The file to use for self-checking, any changes here should only be to adding more problems for you to check (the submission won't check your added problems).
3. Pressure_plate.py – The file that holds the class of the game assignment, it is encouraged to look into it and see how it operates.

Submission:

- Deadline – 12.6.25
- Solo submissions.
- Please put your ID in the variable id in ex2.py and along with submitting ex2.py please submit a details.txt that includes your name (in English only) and your ID (One line after the other).
- Any questions please use the Lamda forum for the assignment.
- The submission will use the [submit](#) system. Upload only the ex2.py and details.txt. Do not upload any other files or in any other format (like zip or rar).